SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – CONCEPT MAPPING

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 2 – Concept Mapping
Weightage:	20% of CIA	**Total Marks:**	100 (2 Questions × 50 Marks)
CO1 Statement:	*Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)*		
Student Name:	ADHITI. S	**Reg. No.:**	192411069
Section / Batch:	BE CSE	**Date:**	20-07-2026

Instructions to Students
- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Draw a clear and neat concept map for each question.
- Identify all key concepts and arrange them in a logical hierarchy.
- Show meaningful linkages between concepts using arrows and connecting words.
- Compare related concepts wherever required and justify the relationships clearly.
- Ensure the concept map is well-organized, readable, and properly labelled.

Question Type	Focus Area	Questions	Marks
Concept Mapping	Map algorithm efficiency concepts using asymptotic analysis, search techniques, recurrence relations, and recursion tree method	Q1 – Q2	Q1 –50 Q2 –50
	GRAND TOTAL:	**100 Marks**	

SECTION A – CONCEPT MAPPING

Q1). Consider the concept map : Algorithm → Time Complexity → Asymptotic Notation → { Big-O, Big-Theta, Big-Omega } Map the relations between each node and explain how the flow leads from an abstract algorithm to measurable efficiency metric. Extend the map to include Best-Case, Worst-Case & Average-Case Analysis.

Q41). Consider the concept map: Sorting Algorithms → Merge sort → Divide → Conquer → Combine → O(n log n) Explain how merge sort works through divide-and-conquer. Extend the map by linking recurrence relation. Justify how the recurrence leads to final complexity. Interpret why merge sort is efficient.

Code of Conduct

I ADHITI. S (192411069) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Adhiti.S

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Marks Evaluation:

Question No.	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)	Total Marks (Each – 50)
Q1	4	3	4	4	3	46
Q2	4	3	4	3	4	44
					Overall Total (100)	90

Faculty In-charge	Course Coordinator	Head of Department
Signature:	Signature:	Signature:
Date: 20/7/26	Date:	Date:

ALGORITHM

Step - by - step procedure to solve a computational problem

Note : Big - O Notation provides an UPPER BOUND on an algorithm's growth rate.

Note : Two algorithms solving the same problem can have DIFFERENT time complexities

Time Complexity

Definition

- Running time of an algorithm
- Growth with input size (n)
- Machine Independent
- Used to compare algorithms

Case Analysis

Best	Average	Worst
• Minimum execution time	• Expected running time	• Maximum execution time
• Most favourable input	• Random input	• Least favourable input
• Lower bound	• Practical performance	• Performance guarantee
• Eg: LINEAR SEARCH O(1)	• Eg: LINEAR SEARCH O(n)	• Eg: LINEAR SEARCH O(n)

Asymptotic Notation

Big - O (O)	Big - Theta (Θ)	Big - Omega (Ω)
• Upper bound	• Tight bound	• Lower bound
• Maximum growth	• Exact growth rate	• Minimum growth
• WORST CASE	• AVERAGE CASE	• BEST CASE
• Eg: Merge Sort O(n log n)	• Eg: Merge Sort Θ(n log n)	• Eg: Insertion Sort Ω(n)

Space Complexity

- Memory Usage
- Auxiliary Space
- Input Space

SORTING ALGORITHMS

Algorithms used to arrange data in ascending / descending order

Time Complexity:
Best | Average | Worst
$= O(n \log n)$

Space Complexity:
→ $O(n)$
→ Auxiliary Memory

Merge Sort

- Comparison based Sorting.
- Stable algorithm
- Recursive algorithm
- Divide and Conquer technique
- efficient for large datasets

Working Principle

DIVIDE	CONQUER	COMBINE
splits array into • 2 equal halves • continue until 1 elt remains • Base case: Single elt is already sorted.	recursively sorts • left sub-array • right sub-array • Recursive function calls	merges • Compare elements • Merge in sorted order. • Final sorted array

Recurrence Relation

Represented by:

$T(n) = 2T(n/2) + n$

- $2T(n/2)$ +n
  - Sorting left half
  - Sorting right half
  - Time required for merging

Master Theorem

Solves the recurrence
- $a = 2$
- $b = 2$
- $f(n) = n$
- case 2 applies

↓

$\Theta(n \log n)$

Efficiency

- Balanced division of problem.
- Logarithmic recursion depth
- Linear-time merging
- Guaranteed $O(n \log n)$
- consistent performance.